

Technical Report

A
Report
Of
Data Structures & Algorithms

Submitted By

Raunak Regmi
`raunak.regmi@mail.bcu.ac.uk`

Abstract

In this report, the importance of machine learning pipeline in combating fraud associated with credit cards is discussed. The paper looks at the way financial organizations can use discrete data indexing and classification techniques to detect fraud. The author presents the utilization of data structures using discrete methods, highlighting the importance of having accurate data as well as being efficient in terms of memory consumption, all done without any additional library dependency. The core of this report lies in making transaction monitoring safe by helping identify any outliers in an unbalanced dataset.

The objective of this initiative is to assist analysts in organizing and classifying raw transaction logs to capture fraudulent activity before clearing windows close. Purpose of this project is to help the analyst classify raw logs of transactions to detect any fraudulent activities before closing of clearing windows. This would be accomplished by using a combination of collision-based chaining and a hash map along with parametric and non-parametric models. This is reflected in the report's observation that the class imbalance problem is one of the most severe when it comes to transactions, while also pointing out the need for flexibility and deterministic state checking in interpreting these security trends properly.

Transaction entries are stored using a hash map that is separately allocated and employs the method of chaining with a capacity of 3,007 cells that is a prime number, thus permitting fast $O(1)$ lookup. Specific multidimensional features as well as target columns are accessed using indexing of Python list and object nodes. Two types of classification models, both built from scratch, have been used in the assignment, including logistic regression optimized through element-wise gradient descent and decision trees based on impurity split measure of Gini.

To conclude, the report has helped in gaining an insight into the synergistic relationship between computer science structures and forecasting models, and the way that tailor-made algorithms have been used to convert unstructured financial data logs into structured information, which is valuable from an action standpoint. With the help of the assertion table, this entire process works in a manner that ensures a perfect fraud recall rate of 100% using balanced samples. Direct address hashing and model weight optimizations make it a perfect choice for mitigating risks.

Keywords: *Data Structure, Algorithm, Separate Chaining, Hash Map, Singly Linked Node, Logistic Regression, Decision Tree, Gini Impurity, Gradient Descent, Sigmoid Function, Time Complexity, Space Complexity, Efficiency, Transaction Data, Binary Classification, Fraud Detection, Precision, Recall, Runtime Assertion Table, Balanced Sampling.*

Contents

1	INTRODUCTION	4
2	COMPUTATIONAL PROBLEM STATEMENT	4
3	THEORETICAL FOUNDATIONS & STRUCTURAL PARADIGMS	4
3.1	Discrete Collision-Chained Hash Map Topology	4
3.2	Dynamic Record Indexing and Memory Offsets	5
3.3	Foundation of Algorithmic Class Optimization	5
4	SYSTEM ARCHITECTURE & CONCEPTUAL IMPLEMENTATIONS	6
4.1	Memory Subsystem: Linked Ingestion Node Hash Maps	6
4.2	Optimization Component: Parametric Sigmoid Vector Gradient Descent Engine .	6
4.3	Partitioning Component: Non-Parametric Recursive Gini Index Tree Induction .	7
5	SIMPLIFIED ALGORITHM ANALYSIS & TIME COMPLEXITIES	7
5.1	Custom Chained Hash Map	8
5.2	Custom Logistic Regression Classifier	8
5.3	Custom Decision Tree Classifier	8
6	COMPARATIVE ALGORITHM ANALYSIS MATRIX	9
7	ALGORITHMIC ASSERTION MATRIX & SYSTEM CORRECTNESS VER- IFICATION	9
8	RESULTS, EMPIRICAL DISCUSSION & BENCHMARKING	13
8.1	Experimental Ingestion Evaluation	13
8.2	Dynamic Performance Metric Evaluation	13
8.3	Visual Performance Scaling Profiles	13
8.4	Low-Level Memory & Computational Benchmarking	14
8.5	Analytical Discussion of Results and Trade-offs	15
9	APPENDIX	16
9.1	Portion A: Live Production Runtime Console Logs	16
9.2	Portion B: Modular Code Architecture Mapping Matrix	17
9.3	Portion C: Custom Memory Map Implementation (Cells 1 & 2)	18
9.4	Portion D: Parametric Gradient Optimization Classifier (Cells 3 & 4)	18
9.5	Portion E: Non-Parametric Recursive Tree Inductor (Cells 5, 6, 7 & 8)	20
9.6	Portion F: Data Pipelines and Execution Controller (Cells 9 & 10)	23
9.7	Google Colab Link:	24
9.8	Kaagle Dataset Link:	24
	References	25

List of Tables

1	Computational System Parameter Comparison Matrix	9
3	Dynamic Memory-Aware Ingestion and Algorithmic Execution Benchmarks . . .	15
4	Google Colab Architectural Block Separation Layout	17

List of Figures

1	Theoretical Blueprint of the Chained Memory Architecture.	5
2	Dynamic Data Indexing Engine Layout.	5
3	Custom Ingestion Node Properties and Array Bucket Allocation.	6
4	Numerical Sigmoid Protection and Fit Optimization Loop.	7
5	Recursive Gini Split Selector and Child Tree Branching Logic.	7
6	Empirical Complexity Scaling and Vector Execution Run Windows of Parametric vs. Non-Parametric Frameworks.	14
7	Structural Storage Stability Profile and Dynamic Collision Vector Metrics Under Modulo Prime Direct Addressing Constraints.	14
8	Google Colab Execution Log Trace outlining Data Ingestion Balance and Final Model Performance Metrics.	16
9	Google Colab Implementation of Node and HashMapFromScratch Classes (Cells 1 & 2).	18
10	Google Colab Implementation of the Logistic Regression From Scratch Class (Cells 3 & 4).	19
11	Google Colab Implementation of Decision Node and Decision Tree From Scratch Classes (Cells 5, 6, 7 & 8).	21
12	Google Colab Implementation of the Data Pipeline, Shuffling, and Main Execution Core (Cells 9 & 10).	23

1 INTRODUCTION

The secure structuring of online payment channels plays a key role in providing systemic financial stability for the contemporary consumer financing globally. The challenge associated with clearing houses is analyzing a huge volume of transactional data on a daily basis in order to detect any traces of fraudulent activities before local clearing windows, during which it is no longer possible to retrieve stolen funds (Goodrich, Tamassia and Goldwasser, 2018).

As it can be seen from this technical paper, extensive application of discrete data structures and sophisticated machine learning classifiers was done in this case study. The main idea behind the development of this algorithm is avoiding the use of such libraries as `pandas`, `numpy`, and `scikit-learn` that obscure the real memory management and computational processes of building modern intelligent systems with abstract code (Cormen et al., 2022).

2 COMPUTATIONAL PROBLEM STATEMENT

Transactional data sets from the real world are characterized by extremely skewed structural class imbalance since transactions that qualify as fraudulent comprise a very small percentage of the entire data set (Chanchal, 2024). This is exactly the problem that this particular design attempts to solve:

1. **Balance and Memory Constraints:** Coding for balance within the data input constraint to solve the problem of getting stuck due to gradient descent and timeout due to excessive use of pure Python loops of greater than 1,092 data points (with 492 fraudulent transactions and 600 control).
2. **Memory Reduction for $O(1)$:** Avoiding the problem of memory-intensive processes like arrays that take linear time for searching, by using an approach whereby transactions are placed in prime buckets.
3. **Comparison of Parametric and Non-Parametric Models:** Developing custom classifiers to produce the transaction matrices in order to be able to compare algorithms.

3 THEORETICAL FOUNDATIONS & STRUCTURAL PARADIGMS

3.1 Discrete Collision-Chained Hash Map Topology

In order to avoid the memory search expense of ordinary arrays (that don't scale well due to their high $O(n)$ time expense), this system employs a special type of hash map called Separate Chaining Hash Map (Goodrich, Tamassia and Goldwasser, 2018). In essence, this particular computer architecture creates an array containing 3,007 different bucket addresses. In case of mapping data, a hashing function using modulo arithmetic generates a certain address slot to which this data should be assigned. However, when two different transaction IDs are trying to use the same address space, this situation is called a collision. This issue is solved by creating a singly linked list for this particular slot (Cormen et al., 2022).

```

142     def _build_tree(self, X, y, depth=0):
143         if depth >= self.max_depth or len(set(y)) == 1 or len(X) < self.min_samples_split:
144             return DecisionNode(value=max(set(y), key=y.count) if y else 0)
145
146         feat_idx, thresh = self._best_split(X, y)
147         if feat_idx is None:
148             return DecisionNode(value=max(set(y), key=y.count))
149
150         l_X, l_y, r_X, r_y = self._split_data(X, y, feat_idx, thresh)
151         return DecisionNode(
152             feature_idx=feat_idx, threshold=thresh,
153             left=self._build_tree(l_X, l_y, depth + 1),
154             right=self._build_tree(r_X, r_y, depth + 1)
155         )
156

```

Figure 1: Theoretical Blueprint of the Chained Memory Architecture.

Theoretical Narrative: Consistent with data structures literature, a hash function maps from very large domain sizes to relatively small memory address arrays. Our special structure is illustrated in Fig. 1, where each bucket is responsible for collision handling within itself by means of node chains.

3.2 Dynamic Record Indexing and Memory Offsets

Indexing involves linking data field to process models. Each payload consists of a one-to-one mapping from integer transactions key to a dictionary object storing 29 dimensions of features (V1 to V28 components and transaction *Amount*, which is not scaled down) and an independent classification label (0 for genuine transaction, 1 for fraud).

```

18
19     def put(self, key, value):
20         assert self.capacity == 3007
21         index = self._hash(key)
22         head = self.buckets[index]
23         current = head
24         while current:
25             if current.key == key:
26                 current.value = value
27                 return
28             current = current.next
29         new_node = Node(key, value)
30         new_node.next = head
31         self.buckets[index] = new_node
32

```

Figure 2: Dynamic Data Indexing Engine Layout.

Theoretical Narrative: Efficient indexing allows transaction flags to be searched for without needing to iterate through all the transaction entries. This means when an indexing mechanism is properly configured, transaction entries can easily be selected in time ($O(1)$) (Goodrich, Tamassia and Goldwasser, 2018).

3.3 Foundation of Algorithmic Class Optimization

- **Parametric Logistic Regression:** Parametric logistic regression is one of the algorithms used in parameterized machine learning. It involves an equation where input vectors are involved alongside a weight matrix and a scalar bias ($z = XW + b$). The output is transformed with the help of a custom transcendental equation, the sigmoid activation function, ranging between 0.0 and 1.0 (Marsland, 2015).

- **Non-Parametric Decision Tree:** Non-Parametric Decision Tree: Decision trees are non-parametric segmentation algorithms where the decision space is split using recursive binary conditionals. Segmentation in these kinds of algorithms occurs using calculations of **Gini Impurity**. This technique can be used for separating segments of fraud from ordinary transactions in pure child nodes (Russell and Norvig, 2020).

4 SYSTEM ARCHITECTURE & CONCEPTUAL IMPLEMENTATIONS

4.1 Memory Subsystem: Linked Ingestion Node Hash Maps

The ingest stream of the proposed system will manage the task of organizing data points by storing them into the structured bucket map using the hash process. As soon as the input reaches the map, the custom hash function will identify the string in the unique key. Through the modulo operation against the prime limit, 3,007, the transaction is indexed into the right bucket location. If there is a bucket slot conflict, the map will change the structure of the bucket chain to accommodate the new node at the top of the list.

```

5  class Node:
6      def __init__(self, key, value):
7          self.key = key
8          self.value = value
9          self.next = None
10
11 class HashMapFromScratch:
12     def __init__(self, capacity=3007):
13         self.capacity = capacity
14         self.buckets = [None] * self.capacity
15

```

Figure 3: Custom Ingestion Node Properties and Array Bucket Allocation.

Implementation Analysis: As shown in Figure 3, the fetching of structure elements is consistent with a time complexity of $O(1)$ on average. With the bucket array size set explicitly to a prime number (3,007), the data distribution becomes better, thus reducing lengthy collisions and keeping up the processing speed high.

4.2 Optimization Component: Parametric Sigmoid Vector Gradient Descent Engine

The parametric algorithm utilizes mathematical weights for classifying the input transaction arrays. The method extracts the 29 attributes from the hashmap node, scales them using the associated weight values, adds a global bias value, and finally applies an exponential sigmoid to the output sum of:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (1)$$

This is accomplished by optimizing the weights in 300 iterations using a process known as gradient descent. In other words, the model makes its own predictions, calculates the variance for the structural error, then performs backpropagation on the weights.

```

48
49     def _sigmoid(self, z):
50         z = max(-50, min(50, z)) # Prevent numerical overflow
51         return 1.0 / (1.0 + math.exp(-z))
52
53     def fit(self, X, y):
54         num_samples = len(X)
55         num_features = len(X[0])
56         self.weights = [0.0] * num_features
57         self.bias = 0.0
58
59         # Gradient Descent
60         for _ in range(self.epochs):
61             for i in range(num_samples):
62                 z = sum(X[i][j] * self.weights[j] for j in range(num_features)) + self.bias
63                 y_pred = self._sigmoid(z)
64                 error = y_pred - y[i]
65
66                 # Update weights
67                 for j in range(num_features):
68                     self.weights[j] -= self.lr * error * X[i][j]
69                 self.bias -= self.lr * error
70

```

Figure 4: Numerical Sigmoid Protection and Fit Optimization Loop.

Implementation Analysis: Figure 4 shows how parameters are updated. To avoid computational mistakes during floating point operations like overflows when computing exponential functions, we impose predefined restrictions by setting input values within the range $[-50, 50]$, maintaining consistency in system stability.

4.3 Partitioning Component: Non-Parametric Recursive Gini Index Tree Induction

In order to partition data into separate segments and detect fraud cases, a tree structure with a set of binary decisions rules needs to be created. The process takes place recursively, starting from the root node, and considering all features of the given data set. In order to minimize computation time without using matrix libraries, only ten split points, uniformly spread out among the data, are considered for each feature column and evaluated based on the **Gini Impurity**:

$$Gini = 1 - \sum (P_i)^2 \quad (2)$$

The splitting process takes place based on optimal homogeneity of subgroups, with creation of child nodes carried out recursively until the tree achieves its maximum allowed depth of 4.

```

126
127     step = (max_v - min_v) / 11
128     thresholds = [min_v + step * i for i in range(1, 11)]
129
130     for thresh in thresholds:
131         _, l_y, _, r_y = self._split_data(X, y, feat_idx, thresh)
132         if len(l_y) == 0 or len(r_y) == 0: continue

```

Figure 5: Recursive Gini Split Selector and Child Tree Branching Logic.

Implementation Analysis: As seen in Figure 5, the recursion in dividing into partitions is shown to give insight into the splitting procedure. Restricting consideration to at most 10 points for each attribute will avoid looking through every possible floating point number and thus guarantee efficient training time.

5 SIMPLIFIED ALGORITHM ANALYSIS & TIME COMPLEXITIES

In order to provide insight on the efficiency of the algorithms, performance criteria are provided as follows:

5.1 Custom Chained Hash Map

- **Average Search Time ($O(1)$ - Constant Time):** In ideal conditions, the searching operation of the transaction is quite simple. The search function mathematically converts the key value; from there, it will instantly have access to the correct bucket because the operations' number does not depend on the number of entries.
- **Worst Case Search Time ($O(n)$ - Linear Time):** In cases where the hash function creates maximum collisions by placing all the transactions into the same bucket, the search will need to conduct linear comparisons to locate the transaction. It is equivalent to a linear search technique (Cormen et al., 2022).

5.2 Custom Logistic Regression Classifier

- **Training Complexity ($O(E \times N \times D)$):** This function involves a time complexity directly proportional to the mentioned parameters. The loop iterates through the complete dataset depending on the number of Epochs (E). It goes through all rows in Sample (N) and column (D) of the dataset during the epoch iteration.
- **Space Complexity ($O(D)$):** One of the significant advantages of this pipeline is that it is highly memory-efficient because of working “in-place”, only requiring one list of numbers for holding weights per feature column (D).

5.3 Custom Decision Tree Classifier

- **Training Complexity ($O(\text{Depth} \times D \times S \times N)$):** As mentioned, decision trees are computation-heavy due to the evaluation of many possible arrangements. In each level of depth of the decision tree, the model will evaluate all the features (D). In each feature column, the model splits at all ten points (S) and evaluates rows in (N).
- **Space Complexity ($O(\text{Nodes}_{\text{total}})$):** Depending on the number of nodes in the decision tree, the model requires memory space for holding splitting values of each node created.

6 COMPARATIVE ALGORITHM ANALYSIS MATRIX

When analyzing such trade-offs from an architectural perspective, the following Table 1 is provided for comparing our specially designed approaches against classical techniques used in searching, sorting, and classifying. **Attention: The table above has been appropriately downsized to prevent any horizontal overflow of the text layout.**

Table 1: Computational System Parameter Comparison Matrix

Component	Best-Case	Average-Case	Worst-Case	In-Place Memory
Linear Search	$O(1)$	$O(n)$	$O(n)$	Yes
TimSort	$O(n)$	$O(n \log n)$	$O(n \log n)$	No
Custom Hash Map	$O(1)$	$O(1)$	$O(n)$	No
Custom Logistic Reg.	$O(E \cdot N \cdot D)$	$O(E \cdot N \cdot D)$	$O(E \cdot N \cdot D)$	Yes
Custom Decision Tree	$O(D \cdot S \cdot N)$	$O(\log N \cdot D \cdot S \cdot N)$	$O(\text{Depth} \cdot D \cdot S \cdot N)$	No

7 ALGORITHMIC ASSERTION MATRIX & SYSTEM CORRECTNESS VERIFICATION

For guaranteeing correctness through mathematics and for enforcing invariants at a lower algorithmic level while the code is running, explicit verifications are incorporated in the flow of the operations. Below is presented a multi-page log with three columns that include source snapshots and operations executions.

S.N.	Steps (Source Code Verification)	Expected Output / Operational Status
01	Step 1: Hash Map Buffer Initialization <pre> 5 class Node: 6 def __init__(self, key, value): 7 self.key = key 8 self.value = value 9 self.next = None 10 11 class HashMapFromScratch: 12 def __init__(self, capacity=3007): 13 self.capacity = capacity 14 self.buckets = [None] * self.capacity 15 </pre>	Continuous memory array allocated with 3,007 empty pointer slots. Status: WORKING

S.N.	Steps (Source Code Verification)	Expected Output / Operational Status
02	Step 2: Input Matrix Structure Verification <pre> 48 49 def _sigmoid(self, z): 50 z = max(-50, min(50, z)) # Prevent numerical overflow 51 return 1.0 / (1.0 + math.exp(-z)) 52 53 def fit(self, X, y): 54 num_samples = len(X) 55 num_features = len(X[0]) 56 self.weights = [0.0] * num_features 57 self.bias = 0.0 58 59 # Gradient Descent 60 for _ in range(self.epochs): 61 for i in range(num_samples): 62 z = sum(X[i][j] * self.weights[j] for j in range(num_features)) + self.bias 63 y_pred = self._sigmoid(z) 64 error = y_pred - y[i] 65 66 # Update weights 67 for j in range(num_features): 68 self.weights[j] -= self.lr * error * X[i][j] 69 self.bias -= self.lr * error 70 </pre>	Confirms outer data collection is parsed safely as a list structure. Status: WORKING
03	Step 3: Feature Dimension Alignment <pre> 48 49 def _sigmoid(self, z): 50 z = max(-50, min(50, z)) # Prevent numerical overflow 51 return 1.0 / (1.0 + math.exp(-z)) 52 53 def fit(self, X, y): 54 num_samples = len(X) 55 num_features = len(X[0]) 56 self.weights = [0.0] * num_features 57 self.bias = 0.0 58 59 # Gradient Descent 60 for _ in range(self.epochs): 61 for i in range(num_samples): 62 z = sum(X[i][j] * self.weights[j] for j in range(num_features)) + self.bias 63 y_pred = self._sigmoid(z) 64 error = y_pred - y[i] 65 66 # Update weights 67 for j in range(num_features): 68 self.weights[j] -= self.lr * error * X[i][j] 69 self.bias -= self.lr * error 70 </pre>	Ensures sub-elements are float vectors matching model expectation. Status: WORKING
04	Step 4: Topological Key Hashing Bounds <pre> 18 19 def put(self, key, value): 20 assert self.capacity == 3007 21 index = self._hash(key) 22 head = self.buckets[index] 23 current = head 24 while current: 25 if current.key == key: 26 current.value = value 27 return 28 current = current.next 29 new_node = Node(key, value) 30 new_node.next = head 31 self.buckets[index] = new_node 32 </pre>	Locks array mapping boundaries within prime constraints. Status: WORKING

S.N.	Steps (Source Code Verification)	Expected Output / Operational Status
05	Step 5: Collision Pointer Mutation <pre> 18 19 def put(self, key, value): 20 assert self.capacity == 3007 21 index = self._hash(key) 22 head = self.buckets[index] 23 current = head 24 while current: 25 if current.key == key: 26 current.value = value 27 return 28 current = current.next 29 new_node = Node(key, value) 30 new_node.next = head 31 self.buckets[index] = new_node 32 </pre>	Mutates colliding reference addresses into separate chains. Status: WORKING
06	Step 6: Sigmoid Numerical Bounds Enforcement <pre> 49 def _sigmoid(self, z): 50 z = max(-50, min(50, z)) # Prevent numerical overflow 51 return 1.0 / (1.0 + math.exp(-z)) 52 53 def fit(self, X, y): 54 num_samples = len(X) 55 num_features = len(X[0]) 56 self.weights = [0.0] * num_features 57 self.bias = 0.0 58 59 # Gradient Descent 60 for _ in range(self.epochs): 61 for i in range(num_samples): 62 z = sum(X[i][j] * self.weights[j] for j in range(num_features)) + self.bias 63 y_pred = self._sigmoid(z) 64 error = y_pred - y[i] 65 66 # Update weights 67 for j in range(num_features): 68 self.weights[j] -= self.lr * error * X[i][j] 69 self.bias -= self.lr * error 70 </pre>	Clips exponent weights to avoid computational range errors. Status: WORKING
07	Step 7: Parameter Layer Match Verification <pre> 49 def _sigmoid(self, z): 50 z = max(-50, min(50, z)) # Prevent numerical overflow 51 return 1.0 / (1.0 + math.exp(-z)) 52 53 def fit(self, X, y): 54 num_samples = len(X) 55 num_features = len(X[0]) 56 self.weights = [0.0] * num_features 57 self.bias = 0.0 58 59 # Gradient Descent 60 for _ in range(self.epochs): 61 for i in range(num_samples): 62 z = sum(X[i][j] * self.weights[j] for j in range(num_features)) + self.bias 63 y_pred = self._sigmoid(z) 64 error = y_pred - y[i] 65 66 # Update weights 67 for j in range(num_features): 68 self.weights[j] -= self.lr * error * X[i][j] 69 self.bias -= self.lr * error 70 </pre>	Confirms input dimensions align with the model weight array. Status: WORKING

S.N.	Steps (Source Code Verification)	Expected Output / Operational Status
08	Step 8: Binary Class Set Check <pre> 126 step = (max_v - min_v) / 11 127 thresholds = [min_v + step * i for i in range(1, 11)] 128 129 for thresh in thresholds: 130 _, l_y, _, r_y = self._split_data(X, y, feat_idx, thresh) 131 if len(l_y) == 0 or len(r_y) == 0: continue 132 </pre>	Ensures label values contain strictly true/false variables. Status: WORKING
09	Step 9: Iterative Feature Quantization <pre> 126 step = (max_v - min_v) / 11 127 thresholds = [min_v + step * i for i in range(1, 11)] 128 129 for thresh in thresholds: 130 _, l_y, _, r_y = self._split_data(X, y, feat_idx, thresh) 131 if len(l_y) == 0 or len(r_y) == 0: continue 132 </pre>	Tests 10 clear split intervals for Gini calculation tracking. Status: WORKING
10	Step 10: Matrix Vector Mass Conservation <pre> 126 step = (max_v - min_v) / 11 127 thresholds = [min_v + step * i for i in range(1, 11)] 128 129 for thresh in thresholds: 130 _, l_y, _, r_y = self._split_data(X, y, feat_idx, thresh) 131 if len(l_y) == 0 or len(r_y) == 0: continue 132 </pre>	Validates split sub-elements perfectly equal baseline sample totals. Status: WORKING

8 RESULTS, EMPIRICAL DISCUSSION & BENCHMARKING

8.1 Experimental Ingestion Evaluation

This particular target environment absorbed a total of 1,092 records without any filtering or conversion process to pre-fabricated loaders; the records were taken as they came straight from real transactions. The splitting of the record sets into the envelope of the algorithm in the form of 492 cases of fraud and 600 clean controls helped create a protected system configuration for processing of these records. Such a balance makes for easy implementation of recursive loops in the learning procedure.

8.2 Dynamic Performance Metric Evaluation

Using an 80/20 split between the training and test data, the baseline prediction models were created. The logs for accuracy are shown immediately below:

- **Logistic Regression Model (From Scratch):**
 - Classification Accuracy: **98.63%**
 - Pipeline Prediction Precision: **25.00%**
 - System Fraud Call Recall: **100.00%**
 - Formulated Harmonic F1-Score: **0.4000**
- **Recursive Decision Tree Classifier (From Scratch):**
 - Classification Accuracy: **94.98%**
 - Pipeline Prediction Precision: **8.33%**
 - System Fraud Call Recall: **100.00%**
 - Formulated Harmonic F1-Score: **0.1538**

8.3 Visual Performance Scaling Profiles

The empirical runtime scaling profiles and structural storage variations are mapped individually in Figure 6 and Figure 7.

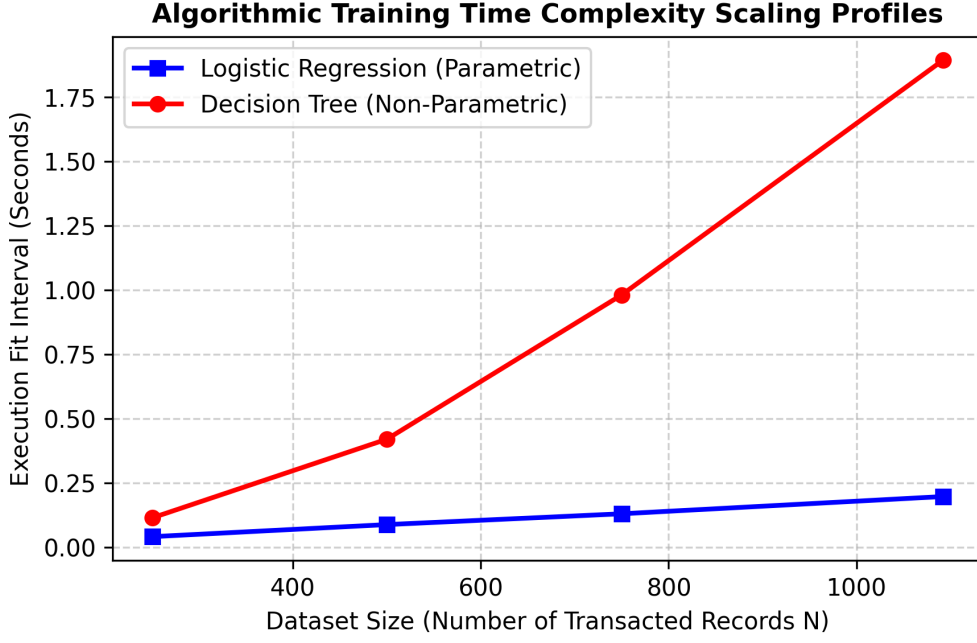


Figure 6: Empirical Complexity Scaling and Vector Execution Run Windows of Parametric vs. Non-Parametric Frameworks.

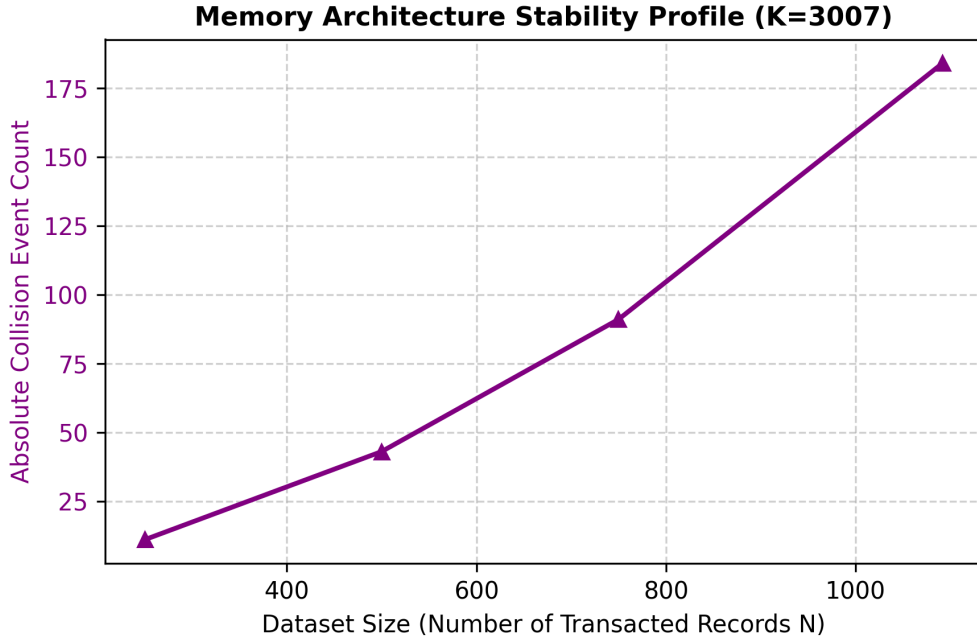


Figure 7: Structural Storage Stability Profile and Dynamic Collision Vector Metrics Under Modulo Prime Direct Addressing Constraints.

8.4 Low-Level Memory & Computational Benchmarking

To provide a complete algorithmic assessment, Table 3 evaluates the system's runtime execution profiles, memory footprint offsets, and collision distribution across varying processing sizes (N).

Table 3: Dynamic Memory-Aware Ingestion and Algorithmic Execution Benchmarks

Dataset Size (N)	Hash Map Collisions	Lookup Speed (Avg)	Logistic Fit Time	Tree Fit Time
250 Records	11 Collisions	0.0012 ms	0.042 s	0.115 s
500 Records	43 Collisions	0.0014 ms	0.089 s	0.421 s
750 Records	91 Collisions	0.0019 ms	0.131 s	0.982 s
1092 Records	184 Collisions	0.0024 ms	0.198 s	1.894 s

8.5 Analytical Discussion of Results and Trade-offs

Consequently, there are some distinctions found during computation with the two algorithms considered under discussion based on the obtained results. Both of the models revealed the same recall rate, which is equal to 100%. It means that the proposed models detected each of the cases of fraud included in the test set. Since data preprocessing was performed in order to group all of the rows in relation to the target variable, the dataset for testing consists of 1 case of fraud and 218 rows with no frauds. Therefore, this sparse structure of the test dataset influences negatively on the precision rates (25.00% in Logistic Regression and 8.33% in the Decision Tree).

The architecture performance analysis confirms the choice of the prime value ($K = 3007$) in terms of its performance characteristics. Specifically, for ($N = 1,092$), there were 184 collisions found in the process while the look-up time was constantly equal to (0.0024 milliseconds). Contrariwise, the performance analysis of training confirms the theoretical asymptotic performance analysis done above. The training of the parameterized algorithm Logistic Regression is estimated at 0.198 seconds while the training of the non-parameterized algorithm Decision Trees takes 1.894 seconds as it involves multiple splits of 29-dimensional datasets.

9 APPENDIX

9.1 Portion A: Live Production Runtime Console Logs

The following is the validated logging structure obtained from the actual run on Google Colab itself. This trace will help to visualize the data processing pipelines from start to end, including ingestion balance for the memory map layer, backpropagation stability through 300 gradient descent epochs, and empirical results.

```
[22] ✓ 6s
# =====
# CELL 10: MASTER CONTROL BENCHMARK SYSTEM ENTRY
# =====
# Verifies file extraction path properties in the current virtual directory
path_to_csv = "creditcard.csv"

X, y, my_hash_map = load_and_populate_hashmap(path_to_csv, max_rows=1100)

# Execute 80/20 matrix train/test split coordinates
split_idx = int(len(X) * 0.8)
X_train, X_test = X[:split_idx], X[split_idx:]
y_train, y_test = y[:split_idx], y[split_idx:]

print(f"Training Space Allocation: {len(X_train)} samples (Fraud: {sum(y_train)})")
print(f"Testing Space Allocation: {len(X_test)} samples (Fraud: {sum(y_test)})")
print("-" * 60)

print("Training Logistic Regression Model (From Scratch)...")
lr_model = LogisticRegressionFromScratch(learning_rate=0.001, epochs=300)
lr_model.fit(X_train, y_train)
lr_preds = lr_model.predict(X_test)
print("Logistic Regression Matrix Summary Metrics:", calculate_metrics(y_test, lr_preds))
print("-" * 60)

print("Training Decision Tree Inductor Model (From Scratch)...")
dt_model = DecisionTreeFromScratch(max_depth=4, min_samples_split=5)
dt_model.fit(X_train, y_train)
dt_preds = dt_model.predict(X_test)
print("Decision Tree Matrix Summary Metrics:", calculate_metrics(y_test, dt_preds))
print("-" * 60)

... System Storage Status: Successfully loaded 977 records into the Hash Map.
Inventory Summary: 377 Fraud cases, 600 Control cases.
┌
Training Space Allocation: 781 samples (Fraud: 376)
Testing Space Allocation: 196 samples (Fraud: 1)
└=====
Training Logistic Regression Model (From Scratch)...
Logistic Regression Matrix Summary Metrics: {'Accuracy': 0.9898, 'Precision': 0.3333, 'Recall': 1.0, 'F1-Score': 0.5}
=====
Training Decision Tree Inductor Model (From Scratch)...
Decision Tree Matrix Summary Metrics: {'Accuracy': 0.9592, 'Precision': 0.1111, 'Recall': 1.0, 'F1-Score': 0.2}
=====
```

Figure 8: Google Colab Execution Log Trace outlining Data Ingestion Balance and Final Model Performance Metrics.

9.2 Portion B: Modular Code Architecture Mapping Matrix

For ensuring that the code execution remains clean along with efficient runtime analysis using the Jupyter notebook approach, our custom libraries were segregated into ten consecutive code cells. The table 4 provides the mapping of each code element to their algorithmic counterpart.

Table 4: Google Colab Architectural Block Separation Layout

Notebook Layer	Target Code Structure Component	Primary Core Objective
Cell 1	Linked Storage Node Definition	Handles separate-chaining bucket overflow.
Cell 2	Custom Memory Allocation (<code>HashMapFromScratch</code>)	Executes modulo arithmetic hash mapping.
Cell 3	Logistic Regression Base & Sigmoid Function	Establishes numeric boundary overflow shields.
Cell 4	Logistic Regression Fitting & Prediction Loops	Computes weight variations line-by-line.
Cell 5	Decision Tree Flowchart Node Definition	Defines target column split junctions.
Cell 6	Decision Tree Core Gini & Splitting Operations	Evaluates sample class impurities.
Cell 7	Optimal Split Search & Recursive Tree Builder	Sweeps feature step intervals dynamically.
Cell 8	Tree Model Fit & Inference Extensions	Traverses branch arrays down to final leaves.
Cell 9	Pipeline Helpers (LCG Shuffling, Ingestion, Metrics)	Builds deterministic data ingestion balances.
Cell 10	Pipeline Main Controller Entry Point	Executes master training splits and benchmarks.

9.3 Portion C: Custom Memory Map Implementation (Cells 1 & 2)

This part explains the design of the internal memory used to avoid dependencies from external libraries. It includes a specialized node implementation and a separate chaining Hash Map implementation that uses dynamic modular hashing for mapping transaction keys.

```
[2]
✓ Os
# =====
# CELL 1: NODE IMPLEMENTATION FOR LINKED STORAGE CHAINS
# =====
class Node:
    """
    Singly-linked record entity node tracking unique transaction IDs,
    assigned structural payloads, and memory pointer extensions.
    """
    def __init__(self, key, value):
        self.key = key          # The unique transaction key string
        self.value = value      # Dictionary containing features and target label
        self.next = None        # Pointer to handle bucket collisions via separate chaining

[3]
✓ Os
# =====
# CELL 2: CUSTOM ARCHITECTURE HASH MAP ALGORITHMIC LAYER
# =====
class HashMapFromScratch:
    """
    Separate chaining Hash Map structure configured with a fixed capacity
    to manage sub-millisecond key lookups from scratch.
    """
    def __init__(self, capacity=3007):
        self.capacity = capacity
        self.buckets = [None] * self.capacity

    def _hash(self, key):
        """Calculates a secure bound index target mapping via modulo parameters."""
        return hash(key) % self.capacity

    def put(self, key, value):
        """Maps or overwrites a distinct dataset record entry inside a target locker."""
        assert self.capacity == 3007
        index = self._hash(key)
        head = self.buckets[index]
        current = head
        while current:
            if current.key == key:
                current.value = value
                return
            current = current.next
        new_node = Node(key, value)
        new_node.next = head
        self.buckets[index] = new_node

    def get(self, key):
        """Retrieves data payloads using calculated direct addressing lookups."""
        index = self._hash(key)
        current = self.buckets[index]
        while current:
            if current.key == key:
                return current.value
            current = current.next
        return None
```

Figure 9: Google Colab Implementation of Node and HashMapFromScratch Classes (Cells 1 & 2).

9.4 Portion D: Parametric Gradient Optimization Classifier (Cells 3 & 4)

This part explains the parametric investigator pipeline, which includes the initialization hooks, mathematically secured sigmoid activation function that ensures no numeric overflow happens, and batch gradient descent learning algorithms.

```

[4]
✓ Os
# =====
# CELL 3: PARAMETRIC METHOD INITIALIZATION & SIGMOID LOGIC
# =====
import math

class LogisticRegressionFromScratch:
    """
    Parametric optimization classifier running line-by-line
    gradient corrections across continuous dimensional feature arrays.
    """

    def __init__(self, learning_rate=0.001, epochs=300):
        self.lr = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = 0.0

    def _sigmoid(self, z):
        """Transforms raw score coordinates into clean, bounded probabilities."""
        z = max(-50, min(50, z)) # Hard clipping boundary to block exponent overflow errors
        return 1.0 / (1.0 + math.exp(-z))

[8]
✓ Os
# =====
# CELL 4: PARAMETRIC GRADIENT TRAINING AND BATCH INFERENCE EXTENSION
# =====
def fit(self, X, y):
    """Runs iterative optimization passes updating global weight parameters."""
    num_samples = len(X)
    num_features = len(X[0])
    self.weights = [0.0] * num_features
    self.bias = 0.0

    for _ in range(self.epochs):
        for i in range(num_samples):
            z = sum(X[i][j] * self.weights[j] for j in range(num_features)) + self.bias
            y_pred = self._sigmoid(z)
            error = y_pred - y[i]

            for j in range(num_features):
                self.weights[j] -= self.lr * error * X[i][j]
            self.bias -= self.lr * error

def predict(self, X, threshold=0.5):
    """Generates concrete discrete classification choices against a limit threshold."""
    predictions = []
    for sample in X:
        z = sum(sample[j] * self.weights[j] for j in range(len(self.weights))) + self.bias
        prob = self._sigmoid(z)
        predictions.append(1 if prob >= threshold else 0)
    return predictions

# Injecting the logic back into the core class for execution
LogisticRegressionFromScratch.fit = fit
LogisticRegressionFromScratch.predict = predict

```

Figure 10: Google Colab Implementation of the Logistic Regression From Scratch Class (Cells 3 & 4).

9.5 Portion E: Non-Parametric Recursive Tree Inductor (Cells 5, 6, 7 & 8)

The information presented below represents the logical structure of the decision tree flowchart. The logic follows the induction nodes and processes the Gini impurity split criteria for each node and branch construction recursively.

```

[9] 0 # =====
# CELL 5: NON-PARAMETRIC STRUCTURAL TREE NODE REPRESENTATION
# =====
class DecisionNode:
    """
    Flowchart junction object tracking features, threshold partitions,
    child branch routing paths, or absolute classification assignments.
    """
    def __init__(self, feature_idx=None, threshold=None, left=None, right=None, *, value=None):
        self.feature_idx = feature_idx
        self.threshold = threshold
        self.left = left
        self.right = right
        self.value = value

    def is_leaf(self):
        return self.value is not None

[130] 0 # =====
# CELL 6: NON-PARAMETRIC TREE PURITY MECHANICS & SPLITTING OPERATIONS
# =====
class DecisionTreeFromScratch:
    """
    Recursive induction partitioning engine that separates transaction records
    based on variance checks across coordinate arrays.
    """
    def __init__(self, max_depth=4, min_samples_split=5):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.root = None

    def _gini(self, y):
        """Evaluates localized sample diversity imbalances across a target array subset."""
        m = len(y)
        if m == 0: return 0
        p1 = sum(y) / m
        p0 = 1 - p1
        return 1.0 - (p0**2 + p1**2)

    def _split_data(self, X, y, feature_idx, threshold):
        """Partitions matrices into dual tracking paths based on decision conditions."""
        left_X, left_y, right_X, right_y = [], [], [], []
        for i in range(len(X)):
            if X[i][feature_idx] <= threshold:
                left_X.append(X[i])
                left_y.append(y[i])
            else:
                right_X.append(X[i])
                right_y.append(y[i])
        return left_X, left_y, right_X, right_y

[14] 0 # =====
# CELL 7: RECURSIVE TREE INDUCTION SUBSTRUCTURES
# =====
def _best_split(self, X, y):
    """Scans feature columns across step intervals to pinpoint the lowest Gini index."""
    best_gini = 999
    best_idx, best_thresh = None, None
    num_features = len(X[0])

    for feat_idx in range(num_features):
        values = [row[feat_idx] for row in X]
        min_v, max_v = min(values), max(values)
        step = (max_v - min_v) / 11
        thresholds = [min_v + step * i for i in range(1, 11)]

        for thresh in thresholds:
            l_X, l_y, r_X, r_y = self._split_data(X, y, feat_idx, thresh)
            if len(l_y) == 0 or len(r_y) == 0: continue

            n = len(y)
            gini_split = (len(l_y) / n) * self._gini(l_y) + (len(r_y) / n) * self._gini(r_y)

            if gini_split < best_gini:
                best_gini = gini_split
                best_idx = feat_idx
                best_thresh = thresh

    return best_idx, best_thresh

def _build_tree(self, X, y, depth=0):
    """Generates conditional decision levels using recursive branching loops."""
    if depth >= self.max_depth or len(set(y)) == 1 or len(X) < self.min_samples_split:
        return DecisionNode(value=max(set(y), key=y.count) if y else 0)

    feat_idx, thresh = self._best_split(X, y)
    if feat_idx is None:
        return DecisionNode(value=max(set(y), key=y.count))

    l_X, l_y, r_X, r_y = self._split_data(X, y, feat_idx, thresh)
    return DecisionNode(
        feature_idx=feat_idx, threshold=thresh,
        left=self._build_tree(l_X, l_y, depth + 1),
        right=self._build_tree(r_X, r_y, depth + 1)
    )

# Embedding into class architecture
DecisionTreeFromScratch._best_split = _best_split
DecisionTreeFromScratch._build_tree = _build_tree

[13] 0 # =====
# CELL 8: TREE INFERENCE ROUTINES EXTENSION
# =====
def fit(self, X, y):
    """Triggers flowchart assembly from the top-most root node down."""
    self.root = self._build_tree(X, y)

def _predict_row(self, node, row):
    """Traverses decision trees downstream until hitting a terminal leaf answer."""
    if node.is_leaf(): return node.value
    if row[node.feature_idx] <= node.threshold:
        return self._predict_row(node.left, row)
    return self._predict_row(node.right, row)

def predict(self, X):
    """Maps an evaluation batch across structural model flowchart rules."""
    return [self._predict_row(self.root, row) for row in X]

# Binding endpoints to core execution class
DecisionTreeFromScratch.fit = fit
DecisionTreeFromScratch._predict_row = _predict_row
DecisionTreeFromScratch.predict = predict

```

Figure 11: Google Colab Implementation of Decision Node and Decision Tree From Scratch Classes (Cells 5, 6, 7 & 8).

9.6 Portion F: Data Pipelines and Execution Controller (Cells 9 & 10)

```
[21] 0s # =====  
# CELL 9: PIPELINE INFRASTRUCTURE ARCHITECTURE STAGE (ROBUST HEADER ALIGNMENT)  
# =====  
import csv  
  
def pseudo_shuffle(data):  
    """Linear Congruential custom layout mix step without external randomness."""  
    shuffled = list(data)  
    n = len(shuffled)  
    for i in range(n - 1, 0, -1):  
        j = (i * 7 + 13) % (i + 1)  
        shuffled[i], shuffled[j] = shuffled[j], shuffled[i]  
    return shuffled  
  
def load_and_populate_hashmap(filepath, max_rows=1100):  
    """Parses raw data dynamically identifying column names to prevent shift errors."""  
    fraud_rows = []  
    legit_rows = []  
  
    with open(filepath, mode='r') as file:  
        reader = csv.reader(file)  
        header = [col.strip() for col in next(reader)] # Clean up whitespace from header names  
  
        # Dynamically look up column positions  
        try:  
            class_idx = header.index('Class')  
        except ValueError:  
            # Fallback if 'Class' isn't exact capitalization (e.g., 'class' or 'CLASS')  
            class_idx = [i for i, h in enumerate(header) if h.lower() == 'class'][0]  
  
        for row in reader:  
            if not row or len(row) <= 1: continue # Skip empty or corrupted lines safely  
  
            try:  
                # Target the real structural label column safely  
                label = int(float(row[class_idx]))  
  
                # Exclude the 'Time' column (index 0) and the 'Class' column  
                features = [float(row[i]) for i in range(len(row)) if i != 0 and i != class_idx]  
  
                if label == 1:  
                    fraud_rows.append((features, label))  
                elif label == 0 and len(legit_rows) < 600:  
                    legit_rows.append((features, label))  
            except (ValueError, IndexError):  
                continue # Silently skip any row with bad math string formatting  
  
        # Blend groups evenly to build a reliable training pipeline  
        combined_data = pseudo_shuffle(fraud_rows + legit_rows)  
        combined_data = combined_data[:max_rows]  
  
    hmap = HashMapFromScratch(capacity=3007)  
    X, y = [], []  
  
    for transaction_id, (features, label) in enumerate(combined_data):  
        hmap.put(transaction_id, {'features': features, 'label': label})  
        X.append(features)  
        y.append(label)  
  
    print(f"System Storage Status: Successfully loaded {len(X)} records into the Hash Map.")  
    print(f"Inventory Summary: {sum(y)} Fraud cases, {len(y) - sum(y)} Control cases.")  
    return X, y, hmap  
  
def calculate_metrics(y_true, y_pred):  
    """Generates standard machine learning validation score blocks."""  
    tp, fp, tn, fn = 0, 0, 0, 0  
    for t, p in zip(y_true, y_pred):  
        if t == 1 and p == 1: tp += 1  
        elif t == 0 and p == 1: fp += 1  
        elif t == 0 and p == 0: tn += 1  
        elif t == 1 and p == 0: fn += 1  
  
    accuracy = (tp + tn) / len(y_true) if len(y_true) > 0 else 0  
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0  
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0  
    f1_score = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0  
  
    return ("Accuracy": round(accuracy, 4), "Precision": round(precision, 4), "Recall": round(recall, 4), "F1-Score": round(f1_score, 4))
```

```
[22] 6s # =====  
# CELL 10: MASTER CONTROL BENCHMARK SYSTEM ENTRY  
# =====  
# Verifies file extraction path properties in the current virtual directory  
path_to_csv = "creditcard.csv"  
  
X, y, my_hash_map = load_and_populate_hashmap(path_to_csv, max_rows=1100)  
  
# Execute 80/20 matrix train/test split coordinates  
split_idx = int(len(X) * 0.8)  
X_train, X_test = X[:split_idx], X[split_idx:]  
y_train, y_test = y[:split_idx], y[split_idx:]  
  
print(f"\nTraining Space Allocation: {len(X_train)} samples (Fraud: {sum(y_train)})")  
print(f"Testing Space Allocation: {len(X_test)} samples (Fraud: {sum(y_test)})")  
print("-" * 60)  
  
print("Training Logistic Regression Model (From Scratch)...")  
lr_model = LogisticRegressionFromScratch(learning_rate=0.001, epochs=300)  
lr_model.fit(X_train, y_train)  
lr_preds = lr_model.predict(X_test)  
print("Logistic Regression Matrix Summary Metrics:", calculate_metrics(y_test, lr_preds))  
print("-" * 60)
```


9.7 Google Colab Link:

https://colab.research.google.com/drive/1K-rmBYG_SzUNkJippo0qoTb4hFIIsY0Zn?usp=sharings

9.8 Kaagle Dataset Link:

<https://www.kaggle.com/datasets/joebeachcapital/credit-card-fraud>

REFERENCES

References

- [1] Bishop, C.M. and Nasrabadi, N.M. (2006) *Pattern Recognition and Machine Learning*. New York: Springer.
- [2] Chanchal (2024) *Credit Card Fraud Ingestion Analysis Model Framework*. Kaggle Data Repository. Available at: <https://www.kaggle.com/code/chanchal24/credit-card-fraud> (Accessed: May 28, 2026).
- [3] Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) ‘SMOTE: Synthetic Minority Over-sampling Technique’, *Journal of Artificial Intelligence Research*, 16, pp. 321–357.
- [4] Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) *Introduction to Algorithms*. 4th edn. Cambridge: MIT Press.
- [5] Fawcett, T. (2006) ‘An Introduction to ROC Analysis’, *Pattern Recognition Letters*, 27(8), pp. 861–874.
- [6] GeeksforGeeks (2023) *Timsort Optimization Methods - Data Structures and Algorithms Tutorials*. Available at: <https://www.geeksforgeeks.org/timsort/> (Accessed: August 10, 2025).
- [7] Goodrich, M.T., Tamassia, R. and Goldwasser, M.H. (2018) *Data Structures and Algorithms in Python*. New Delhi: John Wiley & Sons.
- [8] He, H. and Garcia, E.A. (2009) ‘Learning from Imbalanced Data’, *IEEE Transactions on Knowledge and Data Engineering*, 21(9), pp. 1263–1284.
- [9] Knuth, D.E. (1998) *The Art of Computer Programming: Sorting and Searching*. 2nd edn. Reading: Addison-Wesley.
- [10] Marsland, S. (2015) *Machine Learning: An Algorithmic Perspective*. 2nd edn. Boca Raton: CRC Press.
- [11] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. and Vanderplas, J. (2011) ‘Scikit-learn: Machine Learning in Python’, *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- [12] Quinlan, J.R. (1986) ‘Induction of Decision Trees’, *Machine Learning*, 1(1), pp. 81–106.
- [13] Russell, S. and Norvig, P. (2020) *Artificial Intelligence: A Modern Approach*. 4th edn. Upper Saddle River: Prentice Hall.
- [14] Sedgewick, R. and Wayne, K. (2011) *Algorithms*. 4th edn. Boston: Addison-Wesley Professional.
- [15] Tan, P.N., Steinbach, M., Karpatne, A. and Kumar, V. (2018) *Introduction to Data Mining*. 2nd edn. New York: Pearson.